

Docker is a platform for **packaging and running applications in isolated environments called containers**. Its biggest benefit is consistency: you package your application together with the runtime, libraries, and dependencies it needs, and then run essentially the same package on a laptop, test server, cloud VM, or production machine.

A useful mental model is:

Docker image = blueprint/package

Docker container = running instance of that package

1. Why Docker exists

Suppose you build a Python application that requires:

- Python 3.12
- Flask
- PostgreSQL client libraries
- specific OS packages
- specific environment variables

It works on your computer, but when someone else runs it, they have Python 3.10 and different system libraries.

You get the classic problem:

"It works on my machine."

Without Docker, the other machine has to reproduce your environment manually.

With Docker, you describe the environment:

```
FROM python:3.12
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

Then build it:

```
docker build -t my-app .
```

And run it:

```
docker run my-app
```

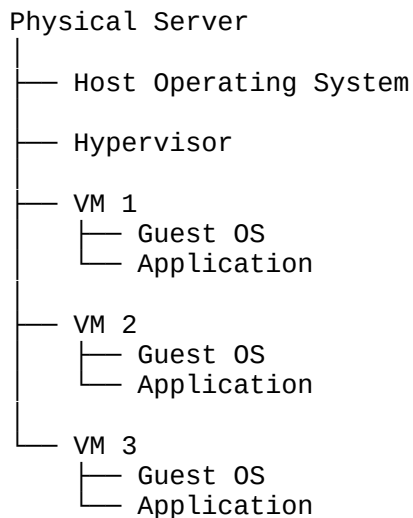
Docker creates an isolated environment containing the application and its dependencies.

2. Containers vs virtual machines

Docker containers are sometimes described as "lightweight virtual machines," but technically they're quite different.

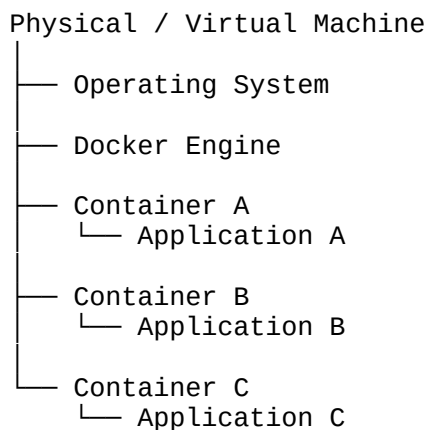
Virtual machine

A VM looks approximately like:



Each VM generally has its own operating-system kernel.

Containers instead look more like:



Containers share the host's underlying kernel while having isolated processes, filesystems, networking, and resource controls.

Because of this, containers are generally much lighter than full VMs.

For example, you might have a VM image measured in gigabytes while a minimal container image might be tens or hundreds of megabytes.

3. The major Docker concepts

Understanding Docker mainly requires understanding these concepts:

```
Dockerfile
↓
docker build
↓
Image
↓
docker run
↓
Container
```

And for sharing:

```
Image
↓
docker push
↓
Registry
↓
docker pull
↓
Another machine
```

Let's examine each one.

4. Dockerfile

A **Dockerfile** is a text file containing instructions Docker uses to construct an image.

Example:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Each instruction has a purpose.

FROM

```
FROM python:3.12-slim
```

Specifies the base image.

Instead of installing Python manually, you're starting from an image that already contains Python.

You could use:

```
FROM ubuntu:24.04
```

or:

```
FROM node:22
```

or:

```
FROM nginx
```

WORKDIR

```
WORKDIR /app
```

Sets the working directory inside the image.

Later commands operate relative to:

```
/app
```

COPY

```
COPY requirements.txt .
```

Copies files from your computer into the image.

Later:

```
COPY . .
```

copies the project into `/app`.

RUN

```
RUN pip install -r requirements.txt
```

Runs a command **while building the image**.

This distinction is important.

RUN:

executed during docker build

while CMD:

executed when the container starts

EXPOSE

EXPOSE 5000

Documents that the application expects to listen on port 5000.

It does **not by itself publish that port to your host**.

CMD

CMD ["python", "app.py"]

Specifies the default command executed when the container starts.

5. Docker images

When you execute:

```
docker build -t my-app .
```

Docker reads your Dockerfile and creates an **image**.

An image is an immutable package containing things such as:

- Operating-system files
- Python runtime
- Libraries
- Dependencies
- Your source code
- Configuration/default startup command

You can see your images with:

```
docker images
```

Example:

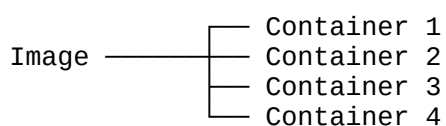
REPOSITORY	TAG	IMAGE ID	SIZE
my-app	latest	abc123def456	180MB
python	3.12	xyz789abc123	1GB

An important concept is that **images are not containers**.

Think:

Class → Object
Image → Container

One image can produce many containers.



6. Docker containers

A container is a running—or stopped—instance of an image.

For example:

```
docker run my-app
```

Docker roughly performs:

1. Find the image
2. Create an isolated container filesystem
3. Configure networking
4. Apply isolation/resource settings
5. Start the container's process

See running containers:

```
docker ps
```

See all containers:

```
docker ps -a
```

Stop one:

```
docker stop <container-name>
```

Remove it:

```
docker rm <container-name>
```

7. What actually runs inside a container?

This is one of the most important Docker concepts.

A container is fundamentally **an isolated process or collection of processes**, not a miniature physical computer.

Suppose:

```
docker run nginx
```

Docker starts nginx as the main process inside an isolated environment.

Conceptually:

```
Host OS
├── Chrome process
├── VS Code process
├── Docker
├── nginx process
│   └── isolated as Container A
└── postgres process
```

isolated as Container B

On Linux, Docker's isolation relies heavily on kernel technologies such as **namespaces** and **cgroups**.

Namespaces isolate things like:

- Processes
- Network interfaces
- Mount points/filesystems
- Hostnames
- Users

Cgroups control/account for resources such as:

- CPU
- Memory
- I/O

So Docker itself didn't invent the fundamental idea of process isolation; it provided an extremely useful system and developer experience around container technologies.

8. Port mapping

Suppose your web application listens on port 5000 inside its container.

```
Container
└─ app
   └─ port 5000
```

Your laptop doesn't automatically expose that service.

You can publish it:

```
docker run -p 8080:5000 my-app
```

Meaning:

```
localhost:8080
    ↓
Container:5000
```

The syntax is:

```
-p HOST_PORT:CONTAINER_PORT
```

So:

```
docker run -p 3000:80 nginx
```

means:

```
localhost:3000
    ↓
container port 80
```

9. Volumes and persistent data

Containers are designed to be disposable.

Imagine PostgreSQL running inside a container:

```
Container
└─ PostgreSQL
   └─ database files
```

If you remove the container, data stored only in its writable container layer can disappear with it.

For databases, that's obviously undesirable.

Docker therefore provides **volumes**.

```
docker run \
  -v postgres-data:/var/lib/postgresql/data \
  postgres
```

Now:

```
Container
└─ /var/lib/postgresql/data
   │
   ▼
  Docker Volume
  postgres-data
```

The container can be deleted and replaced while the volume survives.

This leads to an important principle:

Containers should generally be disposable; important persistent data should live outside the container's ephemeral writable layer.

You can also bind-mount host directories:

```
docker run -v ./src:/app/src my-app
```

This is especially useful during development because changes on your computer immediately appear inside the container.

10. Environment variables

Configuration is commonly passed to containers through environment variables.

For example:

```
docker run \
  -e DATABASE_HOST=db \
  -e APP_ENV=production \
  my-app
```

Inside the container:

```
DATABASE_HOST=db  
APP_ENV=production
```

Applications can read these values at runtime.

Sensitive values such as passwords and API keys require more care; avoid baking them directly into Docker images.

11. Docker registries

How does another computer get your Docker image?

Through a **container registry**.

The most famous public registry is [Docker Hub](#).

Conceptually:

```
Developer  
|  
docker build  
▼  
Local Image  
|  
docker push  
▼  
Registry  
|  
docker pull  
▼  
Production Server
```

For example:

```
docker pull nginx
```

downloads an nginx image.

Then:

```
docker run nginx
```

creates a container from it.

Cloud providers and organizations also operate private registries.

12. Docker Compose

Real applications rarely contain only one service.

You might have:

Frontend
Backend
PostgreSQL
Redis

Running everything manually becomes tedious:

```
docker run ...  
docker run ...  
docker run ...  
docker run ...
```

Docker Compose lets you describe a multi-container application in YAML.

Example:

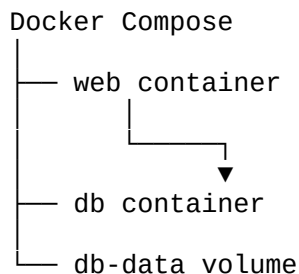
```
services:  
  web:  
    build: .  
    ports:  
      - "8000:8000"  
    environment:  
      DATABASE_HOST: db  
    depends_on:  
      - db  
  
  db:  
    image: postgres:17  
    environment:  
      POSTGRES_PASSWORD: secret  
    volumes:  
      - db-data:/var/lib/postgresql/data  
  
volumes:  
  db-data:
```

Then:

```
docker compose up
```

Docker starts the application stack.

Conceptually:



Compose also creates networking so services can communicate using service names.

For example, your application can connect to:

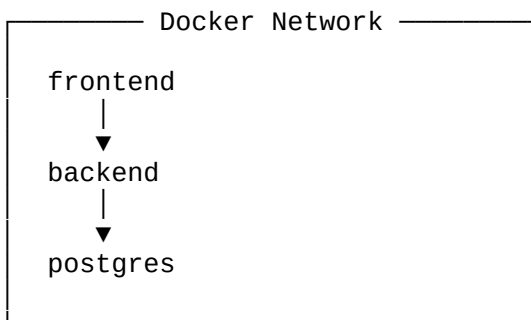
```
db:5432
```

rather than hardcoding an IP address.

13. Docker networking

Containers can communicate through Docker networks.

Suppose Compose creates:



Services can generally resolve each other by their service/container DNS names.

For example:

backend → postgres:5432

Meanwhile, only the ports you publish need to be exposed to the host.

This means your database might be accessible to the backend without being publicly accessible from outside the Docker network.

14. Docker image layers

Docker images are built in **layers**.

Consider:

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

Conceptually:

COPY application code	
Install Python dependencies	
Working directory/config	

Python base image

Docker can cache layers.

This is why Dockerfiles are often written like:

```
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .
```

instead of:

```
COPY . .
RUN pip install -r requirements.txt
```

If you modify one source file, `requirements.txt` probably hasn't changed.

Docker can therefore reuse the cached dependency layer instead of reinstalling everything.

That can dramatically speed up builds.

15. `.dockerignore`

You usually don't want everything in your project copied into the build context.

For example:

```
.git
node_modules
.env
__pycache__
*.log
```

You can create:

```
.dockerignore
```

with:

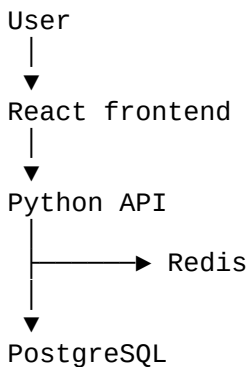
```
.git
node_modules
.env
__pycache__
*.log
```

It serves a role similar to `.gitignore`, but for Docker builds.

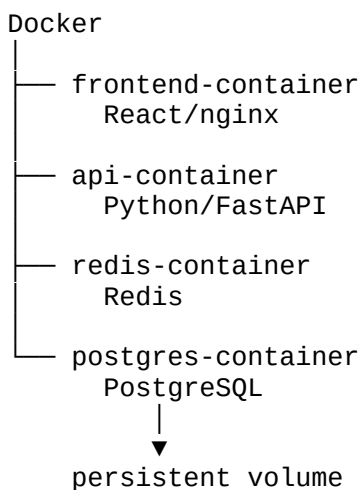
It can reduce build context size and help avoid accidentally including unwanted files.

16. A realistic example

Imagine this architecture:



You could containerize every service:



Compose could start the entire development environment:

```
docker compose up
```

A new developer could clone the repository and potentially get the same services running without manually installing PostgreSQL, Redis, the exact Python version, and so forth.

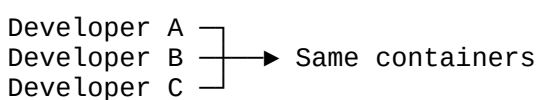
That's one of Docker's biggest advantages.

17. Docker in development vs production

Docker is useful in both, but for somewhat different reasons.

Development

Docker helps developers reproduce environments:



Instead of documentation saying:

```
Install PostgreSQL 17
Install Redis
Install Python 3.12
Install these packages...
Configure this...
```

the project can provide:

```
docker compose up
```

Production

Containers provide standardized deployment units.

A CI/CD pipeline might do:

```
Git push
  ↓
Run tests
  ↓
Build Docker image
  ↓
Push image to registry
  ↓
Deploy image
  ↓
Start containers
```

The image tested by CI can be closely aligned with what gets deployed.

18. Docker and Kubernetes

Docker and Kubernetes are related but are **not the same thing**.

Docker primarily deals with building and running containers.

Kubernetes manages large groups of containerized workloads across machines.

Think:

Docker

"What is my container and how do I run it?"

Kubernetes

"How do I manage hundreds/thousands of container workloads across many machines?"

Kubernetes handles things such as:

```
Scheduling
Scaling
Load balancing
Service discovery
Self-healing
Rolling deployments
Configuration
```

Secrets

For example:

Kubernetes Cluster

Server A	Server B	Server C
App App	App Redis	App App

Modern Kubernetes uses container runtimes through its Container Runtime Interface; Kubernetes itself does not require Docker Engine.

19. Important Docker commands

Once the concepts make sense, most everyday commands are straightforward:

```
# Build an image
docker build -t my-app .

# List images
docker images

# Run a container
docker run my-app

# Run in background
docker run -d my-app

# Publish a port
docker run -p 8080:80 my-app

# Give container a name
docker run --name web my-app

# List running containers
docker ps

# List all containers
docker ps -a

# View logs
docker logs web

# Follow logs
docker logs -f web

# Execute command inside running container
docker exec -it web sh

# Stop container
docker stop web

# Remove container
docker rm web
```

```
# Remove image
docker rmi my-app

# Start Compose application
docker compose up

# Start Compose in background
docker compose up -d

# Stop/remove Compose containers
docker compose down
```

20. The most important distinction: build time vs runtime

Beginners frequently mix these up.

Build time

```
docker build .
```

Docker processes your Dockerfile:

```
FROM
COPY
RUN
...
```

and produces:

```
IMAGE
```

Runtime

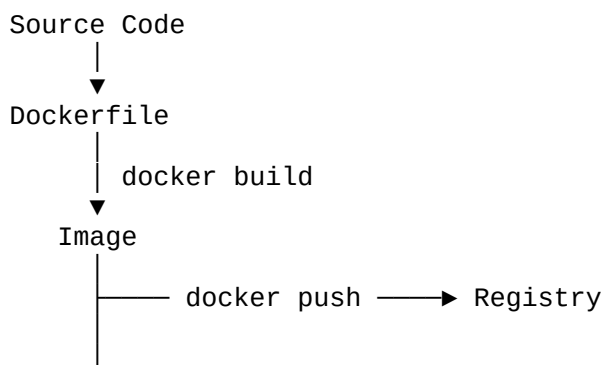
```
docker run my-image
```

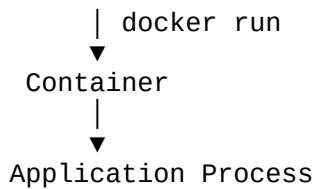
Docker creates:

```
CONTAINER
```

and executes its startup command.

So the complete lifecycle is:





21. What Docker does not do

Docker doesn't magically solve every deployment problem.

It doesn't automatically provide:

- Database backups
- Application security
- Monitoring
- High availability
- Load balancing
- Automatic scaling
- Secrets management
- Production orchestration
- Good application architecture

Docker gives you a standardized way to **package, distribute, isolate, and run software**. Other systems handle the rest.

22. Docker in one practical example

Suppose you've written:

app.py
requirements.txt
Dockerfile

app.py:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Docker!"

app.run(host="0.0.0.0", port=5000)
```

requirements.txt:

flask

Dockerfile:

FROM python:3.12-slim

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app.py .
```

```
CMD ["python", "app.py"]
```

Build:

```
docker build -t hello-docker .
```

Docker creates:

```
hello-docker image
```

```
|
|— Linux userspace/files
|— Python 3.12
|— Flask
|— app.py
```

Run it:

```
docker run -p 5000:5000 hello-docker
```

Now the flow is:

```
Browser
  |
  | localhost:5000
  ▼
Host port 5000
  |
  ▼
Docker
  |
  ▼
Container port 5000
  |
  ▼
Flask application
  |
  ▼
"Hello from Docker!"
```

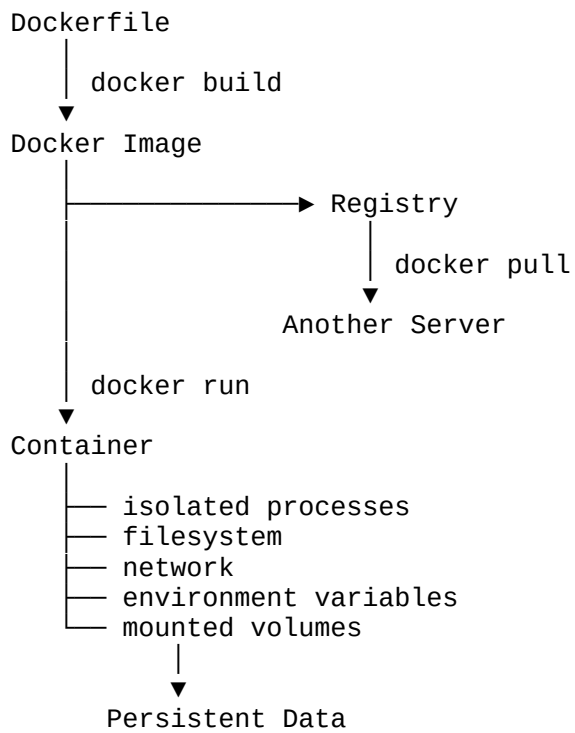
That small example contains most of Docker's core idea.

The big picture

The Docker ecosystem becomes much easier if you remember this pipeline:

DOCKER WORKFLOW

```
Developer
  |
  ▼
Source Code
  |
  ▼
```



The key sentence to remember is:

Docker packages an application and its runtime dependencies into an image, then runs instances of that image as isolated containers.

Once you understand **Dockerfile** → **Image** → **Container**, the next concepts to learn are **ports** → **volumes** → **networks** → **Docker Compose** → **registries** → **production deployment/Kubernetes**.

What is Kubernetes?

Kubernetes (K8s) is an open-source platform for **deploying, managing, scaling, and updating containerized applications**.

The simplest definition is:

Docker runs containers. Kubernetes manages containers at scale.

For example, Docker can run:

Container
└─ Nginx

But imagine you have:

100 containers
10 servers
3 applications
multiple versions
automatic scaling
automatic recovery
rolling deployments
load balancing

Managing all of that manually would be difficult.

Kubernetes automates it.

1. Why do we need Kubernetes?

Suppose you have a web application:

User
↓
Nginx
↓
Backend
↓
PostgreSQL

Initially, you might run:

Backend container 1

But what happens if traffic increases?

You may want:

Backend container 1
Backend container 2
Backend container 3
Backend container 4

And you need traffic distributed between them.

Then suppose container 2 crashes:

Backend 1 
Backend 2 
Backend 3 
Backend 4 

You want Kubernetes to automatically create another one.

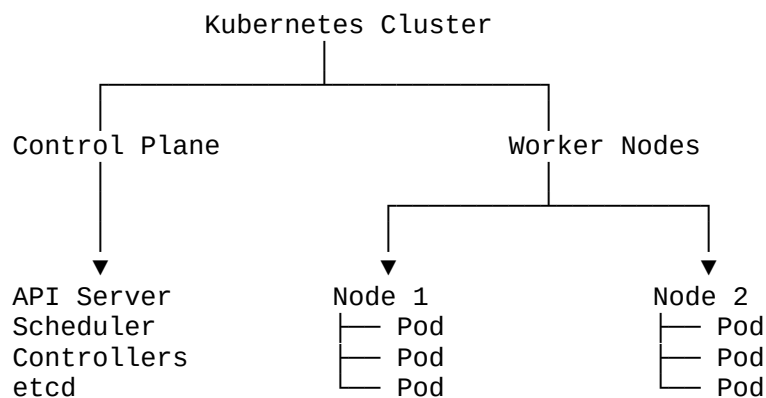
Then imagine one server itself crashes.

You want Kubernetes to move workloads to another server.

That's the type of problem Kubernetes solves.

2. Kubernetes architecture

A Kubernetes cluster generally looks like:



There are two major parts:

Control Plane

The **brain** of Kubernetes.

It decides:

- What should be running?
- Where should it run?
- Should something be restarted?
- Should replicas increase?
- Should a deployment be updated?

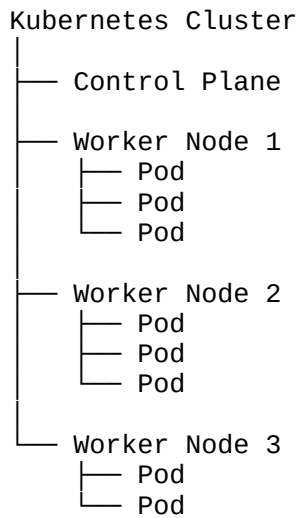
Worker Nodes

The machines where your applications actually run.

3. Kubernetes Cluster

A **cluster** is a collection of machines managed by Kubernetes.

For example:



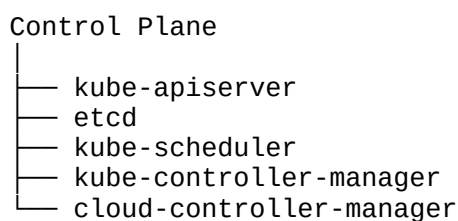
A cluster can be small or very large.

For learning, you can run Kubernetes locally using tools such as **Minikube**, which you've already been working with.

4. Control Plane

The control plane contains the components that manage the cluster.

The most important components are:



Let's understand them individually.

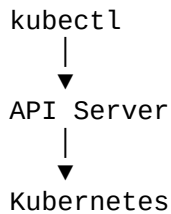
5. kube-apiserver

The **API Server** is the main entry point into Kubernetes.

When you execute:

```
kubectl get pods
```

your request goes approximately:



The API server communicates with the other Kubernetes components.

You can think of it as:

The front door of the Kubernetes cluster.

Almost everything Kubernetes does goes through the API.

6. etcd

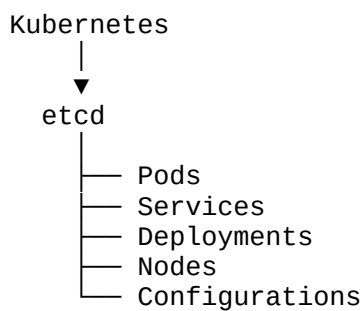
etcd is Kubernetes' distributed key-value datastore.

It stores the cluster's state/configuration.

For example:

What Pods should exist?
What Deployments exist?
What Services exist?
What Nodes exist?
What configuration exists?

Conceptually:



You normally don't directly modify etcd.

Kubernetes components communicate through the API server.

7. kube-scheduler

The scheduler decides:

Which worker node should run this Pod?

Suppose you have:

Node 1
CPU: 80% used

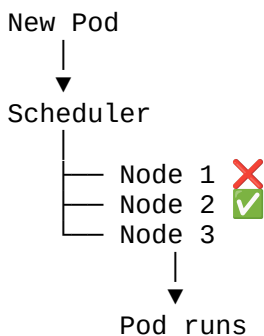
Node 2
CPU: 20% used

Node 3
CPU: 40% used

A new Pod needs to run.

The scheduler evaluates available nodes and scheduling constraints and selects an appropriate node.

Conceptually:



The scheduler considers many factors, including resource requests, constraints, affinity/anti-affinity, taints/tolerations, topology, and more.

8. kube-controller-manager

Controllers continuously compare:

Desired state
vs
Current state

This is one of the **most important Kubernetes concepts**.

Suppose you specify:

replicas: 3

You are saying:

I want 3 replicas.

Kubernetes observes:

Desired:
3 Pods

Current:
2 Pods

The controller notices the difference and works to create another Pod.

Desired = 3
Current = 2

↓

Controller

↓

Create Pod

↓

Desired = 3
Current = 3

This is why Kubernetes is often described as a **declarative system**.

9. Worker Nodes

Worker nodes are the machines that run your workloads.

A node generally contains:

Worker Node

- ├─ kubelet
- ├─ container runtime
- ├─ kube-proxy
- └─ Pods

Let's examine them.

10. kubelet

The **kubelet** is an agent running on each worker node.

Its job is essentially:

Make sure the Pods assigned to this node are running correctly.

For example:

Control Plane

↓ "Run this Pod"

kubelet

↓

Container Runtime

↓

Container

If the container crashes, kubelet/container runtime machinery helps maintain the Pod according to the desired configuration.

11. Container Runtime

Kubernetes needs something capable of actually running containers.

Modern Kubernetes commonly uses runtimes such as:

- containerd
- CRI-O

Historically, Kubernetes commonly interacted with Docker through an integration layer, but modern Kubernetes does **not require Docker Engine** as its container runtime.

So:

```
Kubernetes
  ↓
Container Runtime
  ↓
Container
```

This is an important distinction from the simplified idea:

Kubernetes = Docker

They are not the same thing.

12. kube-proxy

kube-proxy helps implement Kubernetes Service networking on nodes.

It participates in directing network traffic toward the appropriate backend Pods.

Conceptually:

```
Client
  ↓
Service
  ├── Pod 1
  ├── Pod 2
  └── Pod 3
```

The exact networking implementation depends on the cluster's networking stack.

13. Pod — the most important Kubernetes object

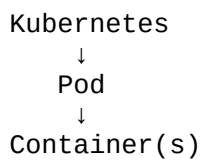
Now we get to one of the most important concepts:

Pod

A Pod is the smallest deployable unit in Kubernetes.

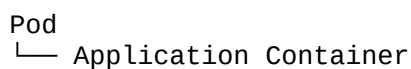
Kubernetes doesn't normally deploy a raw container directly.

Instead:

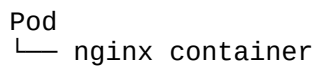


A Pod can contain one or more containers.

Most commonly:



For example:



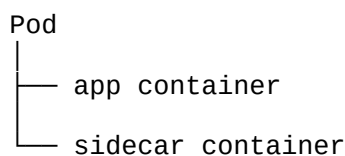
14. Why does Kubernetes use Pods?

A Pod provides a shared execution environment for its containers.

Containers inside the same Pod can share:

- network namespace
- IP address
- localhost
- mounted volumes

For example:



They can communicate using:

localhost

because they share the Pod's network namespace.

However, you generally don't put unrelated applications into one Pod.

Usually:

```
Pod 1 → Backend
Pod 2 → Backend
Pod 3 → Backend
```

is preferable to putting three backend processes into one Pod.

15. Pod IP addresses

Every Pod generally receives an IP address within the cluster's Pod network.

For example:

```
Pod 1 → 10.244.0.5
Pod 2 → 10.244.0.6
Pod 3 → 10.244.0.7
```

But there's a problem.

Pods are **ephemeral**.

A Pod might be deleted and recreated:

```
Old Pod
10.244.0.5
  ↓
deleted
```

```
New Pod
10.244.0.12
```

Therefore, applications shouldn't normally depend on individual Pod IP addresses.

That's where **Services** come in.

16. Deployment

A **Deployment** manages a set of replicated Pods and provides declarative updates.

For example:

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: nginx-deployment
```

```
spec:
  replicas: 3
```

```

selector:
  matchLabels:
    app: nginx

template:
  metadata:
    labels:
      app: nginx

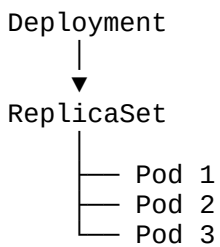
  spec:
    containers:
      - name: nginx
        image: nginx:latest

```

This means approximately:

Create and maintain 3 Pods running nginx.

The architecture becomes:



17. ReplicaSet

A **ReplicaSet** maintains a specified number of Pod replicas.

Suppose:

replicas: 3

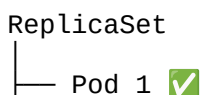
Then:

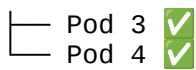


If Pod 2 dies:



The ReplicaSet works with Kubernetes controllers to bring the number back to 3:

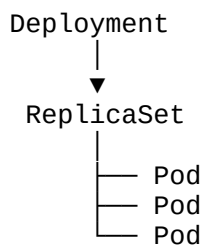




Normally you create a **Deployment** rather than directly managing ReplicaSets.

18. Deployment vs ReplicaSet vs Pod

This hierarchy is very important:



Think:

Deployment

"I want this application deployed and updated this way."

ReplicaSet

"I need this many Pods."

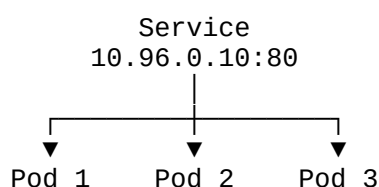
Pod

"Here is the actual workload running on a node."

19. Service

A Kubernetes **Service** provides a stable network endpoint for a group of Pods.

Imagine:



Pods can come and go, but the Service provides a stable abstraction.

For example:

```
apiVersion: v1
kind: Service
```

```
metadata:
  name: nginx-service

spec:
  selector:
    app: nginx

  ports:
    - port: 80
      targetPort: 80
```

The selector:

```
selector:
  app: nginx
```

means:

Send traffic to Pods having the label `app=nginx`.

20. Service types

There are several important Service types.

ClusterIP

Default type.

```
Internet
  X
  |
Cluster
  |
Service
  |
Pods
```

It's primarily accessible inside the cluster.

Useful for:

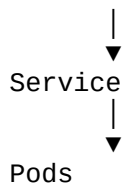
```
Backend ↔ Database
Frontend ↔ Backend
```

NodePort

Exposes a Service through a port on each node.

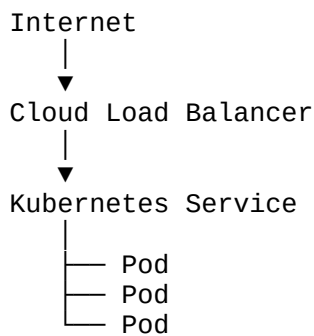
Conceptually:

```
External Client
  |
  ▼
Node IP:30080
```



LoadBalancer

Common in cloud environments.



Cloud providers can provision an external load balancer for this Service type.

21. Labels

Labels are key-value pairs attached to Kubernetes objects.

Example:

```
labels:  
  app: backend  
  environment: production
```

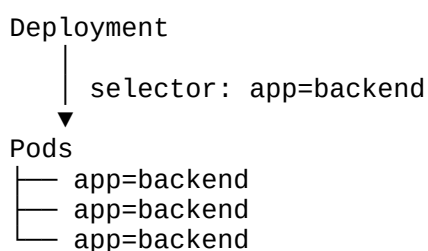
Then another object can select them:

```
selector:  
  app: backend
```

Labels are fundamental to Kubernetes.

They help Kubernetes identify relationships between objects.

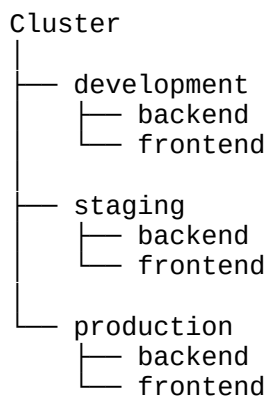
For example:



22. Namespaces

A Kubernetes cluster can be divided logically using **Namespaces**.

For example:



You might create:

```
kubectl create namespace development
```

Then deploy into it:

```
metadata:
  name: backend
  namespace: development
```

Namespaces are useful for organization, access control, resource policies, and separating workloads logically.

23. ConfigMap

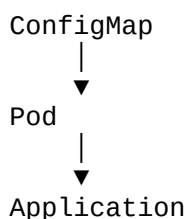
A **ConfigMap** stores non-sensitive configuration.

For example:

```
DATABASE_HOST=postgres
DATABASE_PORT=5432
APP_ENV=production
```

Instead of hardcoding configuration into your Docker image, Kubernetes can provide it to your Pod.

Conceptually:

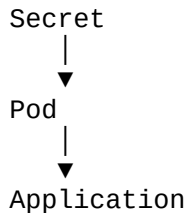


24. Secret

A Kubernetes **Secret** is intended for sensitive configuration such as:

Database password
API token
TLS credentials

For example:

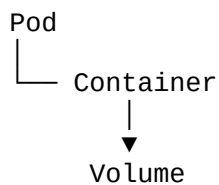


Important: Kubernetes Secrets are **not automatically equivalent to strong encryption at rest** in every deployment. Proper RBAC and encryption-at-rest configuration are important in production.

25. Volumes

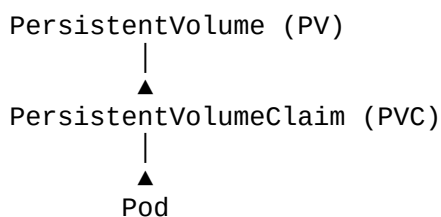
Just like Docker, Kubernetes needs a way to handle storage.

For example:



Kubernetes has several storage abstractions.

One important one is:



A PVC means approximately:

"My application needs persistent storage with these requirements."

This allows storage to be provisioned/bound independently of the Pod lifecycle.

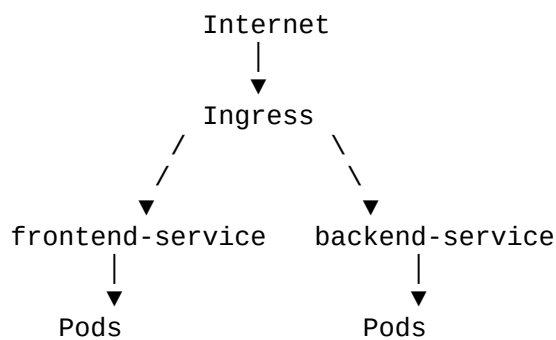
26. Ingress

Suppose you have:

frontend.example.com
api.example.com

You could route external HTTP/HTTPS traffic into different Kubernetes Services using an **Ingress**.

Conceptually:



For example:

example.com
↓
Frontend Service

api.example.com
↓
Backend Service

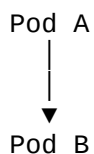
An Ingress requires an **Ingress Controller** to actually implement the routing.

Common implementations include NGINX-based controllers and cloud-provider solutions.

27. Kubernetes networking

Kubernetes networking can initially seem complicated.

A simplified model is:



Pods are expected to be able to communicate over the cluster network according to Kubernetes networking rules.

Then:

Pod
↓
Service

↓
Other Pods

And external traffic might flow:

Internet
↓
Load Balancer / Ingress
↓
Service
↓
Pods

A **CNI plugin** provides much of the cluster networking implementation.

Examples include:

- Cilium
- Calico
- Flannel

The networking details depend on your cluster and CNI.

28. Desired state — the heart of Kubernetes

This is perhaps the single most important Kubernetes concept.

You don't generally tell Kubernetes:

"Create this container, then create another container, then restart this one."

Instead you tell Kubernetes:

"This is the state I want."

For example:

replicas: 3

You're declaring:

I WANT:
3 replicas

Kubernetes observes:

CURRENT:
2 replicas

and works toward:

DESIRED:
3

CURRENT:
3

This is called **reconciliation**.

29. Declarative vs imperative Kubernetes

Imperative

You're telling Kubernetes exactly what action to perform.

For example:

```
kubectl create deployment nginx --image=nginx
```

Declarative

You describe what you want:

```
replicas: 3
image: nginx
```

Then:

```
kubectl apply -f deployment.yaml
```

Kubernetes figures out what actions are necessary.

Declarative configuration is heavily used in production because configuration can be stored in Git and managed through GitOps/CI/CD practices.

30. kubectl

`kubectl` is the command-line tool used to communicate with Kubernetes.

For example:

```
kubectl get nodes
```

Shows nodes.

```
kubectl get pods
```

Shows Pods.

```
kubectl get deployments
```

Shows Deployments.

```
kubectl get services
```

Shows Services.

```
kubectl get all
```

Shows many common resources.

31. Creating a Deployment

Suppose you have:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: nginx-deployment

spec:
  replicas: 2

  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      labels:
        app: nginx

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

Apply it:

```
kubectl apply -f deployment.yaml
```

Then:

```
kubectl get deployments
```

You might see:

NAME	READY	UP-TO-DATE	AVAILABLE
nginx-deployment	2/2	2	2

Check Pods:

```
kubectl get pods
```

Something like:

NAME	READY
nginx-deployment-abc123	1/1
nginx-deployment-def456	1/1

32. Scaling

Suppose you currently have:

2 Pods

You can scale to 5:

```
kubectl scale deployment nginx-deployment --replicas=5
```

Now:

Deployment

```
├── Pod 1
├── Pod 2
├── Pod 3
├── Pod 4
└── Pod 5
```

Or declaratively change:

```
replicas: 5
```

and:

```
kubectl apply -f deployment.yaml
```

33. Self-healing

Suppose:

Deployment

```
├── Pod 1 ✓
├── Pod 2 ✗
└── Pod 3 ✓
```

The desired state is:

3 Pods

Kubernetes reconciles the state:

Pod 2 dies

↓

Kubernetes notices

↓

New Pod created

↓

3 Pods again

This is one of Kubernetes' biggest advantages.

34. Rolling updates

Suppose you're running:

nginx:1.25

and want:

nginx:1.26

A Deployment can perform a rolling update.

Instead of:

✗ Stop everything

then

✓ Start everything

Kubernetes can gradually replace old Pods:

Old version:

Pod 1

Pod 2

Pod 3

↓

Update

↓

Pod 1 → New

Pod 2 → Old

Pod 3 → Old

↓

Pod 1 → New

Pod 2 → New

Pod 3 → Old

↓

Pod 1 → New

Pod 2 → New

Pod 3 → New

This helps reduce downtime.

35. Rollback

If the new version is broken, you can roll back.

For example:

```
kubectl rollout undo deployment nginx-deployment
```

Kubernetes can return to the previous Deployment revision, subject to the Deployment's rollout history and configuration.

36. Health checks

Kubernetes can check whether your application is healthy.

There are three important probe types:

Liveness probe

Asks:

Is this container still alive?

If the liveness check repeatedly fails, Kubernetes can restart the container.

Readiness probe

Asks:

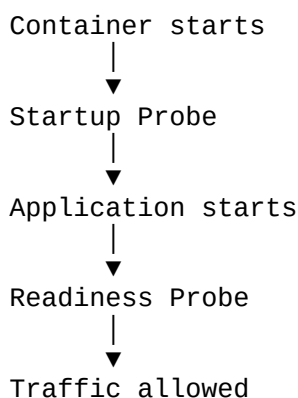
Is this application ready to receive traffic?

If not ready, the Pod can be removed from Service endpoints while it remains running.

Startup probe

Useful for applications that take a long time to start.

Conceptually:



37. Resource requests and limits

You can tell Kubernetes how much CPU and memory your container needs.

Example:

```
resources:
  requests:
```

```
cpu: "250m"
memory: "256Mi"

limits:
  cpu: "500m"
  memory: "512Mi"
```

Meaning roughly:

Request:
CPU → 250 millicores
Memory → 256 MiB

Limit:
CPU → 500 millicores
Memory → 512 MiB

The scheduler uses resource requests when deciding where to place Pods.

38. Horizontal Pod Autoscaler

Kubernetes can automatically change the number of Pod replicas based on metrics.

For example:

Low traffic

↓

2 Pods

Traffic increases

↓

4 Pods

Traffic increases more

↓

8 Pods

This is called **Horizontal Pod Autoscaling (HPA)**.

Conceptually:

CPU utilization

↓

▼
HPA

↓

▼
Deployment

↓

▼
Number of Pods

Actual autoscaling setups depend on metrics availability and configuration.

39. Node failure

Suppose you have:

Node 1
├─ Pod A
└─ Pod B

Node 2
├─ Pod C
└─ Pod D

Node 1 fails:

Node 1 ✖

Node 2
├─ Pod C
└─ Pod D

Kubernetes can detect the node problem and, where appropriate, recreate/reschedule workloads according to their controllers and constraints.

This is one of the reasons Kubernetes is useful for distributed production systems.

40. Kubernetes vs Docker Compose

You've recently been learning Docker Compose, so this comparison is useful.

Docker Compose	Kubernetes
Excellent for local development	Designed for orchestration at scale
Usually single host	Can span multiple nodes
Simple	Much more complex
<code>docker compose up</code>	<code>kubectl apply</code>
Services/containers	Pods/Deployments/Services/etc.
Basic scaling	Advanced scheduling/scaling
Basic recovery	Strong reconciliation/self-healing
Great for development	Common in production platforms
For example:	

Compose

```
services:  
  backend:  
    image: my-backend  
  
  postgres:  
    image: postgres
```

Kubernetes

You might have separate resources:

Deployment
↓
ReplicaSet
↓
Pods

Service
↓
Pods

ConfigMap
Secret
PVC
Ingress

Kubernetes gives you significantly more control, but also significantly more complexity.

41. Docker → Kubernetes relationship

Think about the complete stack:

Application
↓
Dockerfile
↓
Docker Image
↓
Container Runtime
↓
Kubernetes Pod
↓
Deployment
↓
Service
↓
Ingress / Load Balancer
↓
Users

More accurately, Kubernetes manages Pods and relies on a container runtime to execute the containers within those Pods.

42. Your Nginx example

You previously asked about this Deployment:

kind: Deployment

```

apiVersion: apps/v1

metadata:
  name: nginx-deployment
  namespace: nginx

spec:
  replicas: 2

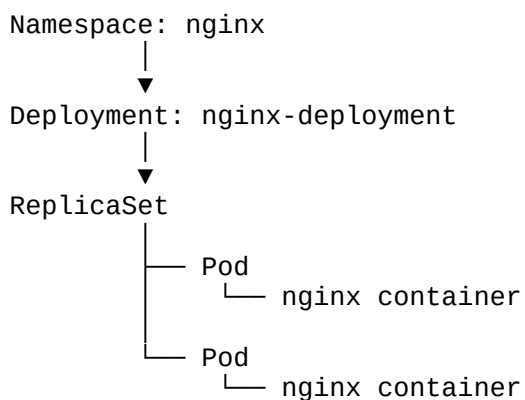
  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      name: nginx-deployment-pod
      labels:
        app: nginx

    spec:
      containers:
        - name: nginx
          image: nginx:latest

```

Now you can understand the hierarchy:



And:

```
replicas: 2
```

means:

Kubernetes should maintain two replicas of this Pod template.

The:

```

selector:
  matchLabels:
    app: nginx

```

connects the Deployment to Pods having:

```

labels:
  app: nginx

```

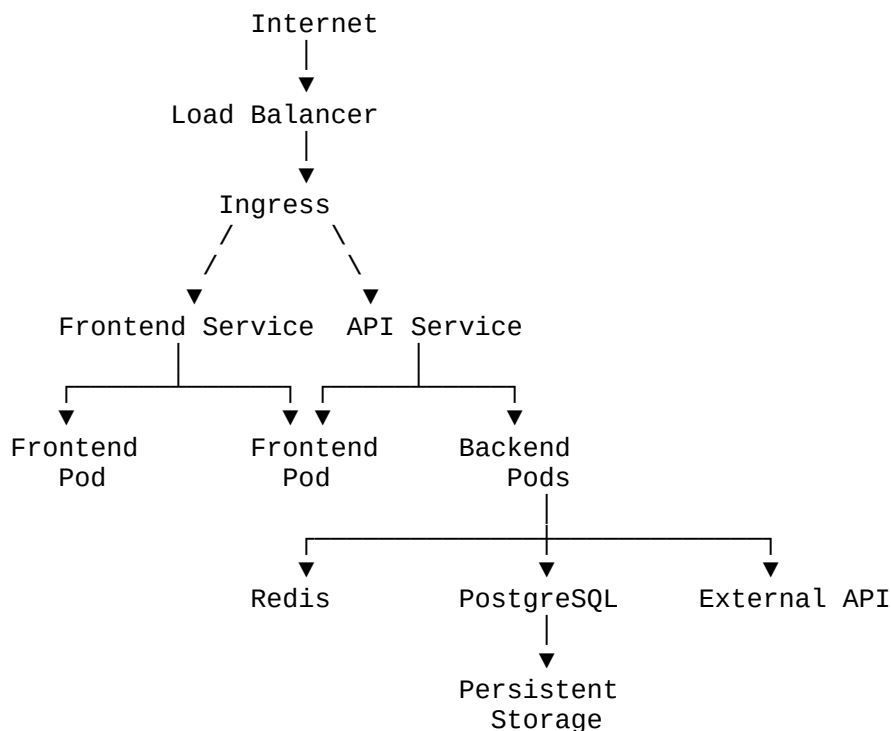
The:

```
image: nginx:latest
```

tells the container runtime which container image to run.

43. A complete production-style architecture

A more realistic web application might look like:



Kubernetes manages much of the orchestration around these workloads.

44. Kubernetes objects you should learn first

Don't try to learn every Kubernetes object immediately.

For your current Docker/DevOps learning path, I recommend this order:

Level 1 — Fundamentals

Cluster
Node
Pod
Container
kubectl
YAML

Level 2 — Application deployment

Deployment
ReplicaSet
Service
Labels

Selectors
Namespaces

Level 3 — Configuration

ConfigMap
Secret
Environment variables

Level 4 — Storage

Volume
PersistentVolume
PersistentVolumeClaim
StorageClass

Level 5 — Networking

ClusterIP
NodePort
LoadBalancer
Ingress
Ingress Controller
DNS
CNI

Level 6 — Reliability/scaling

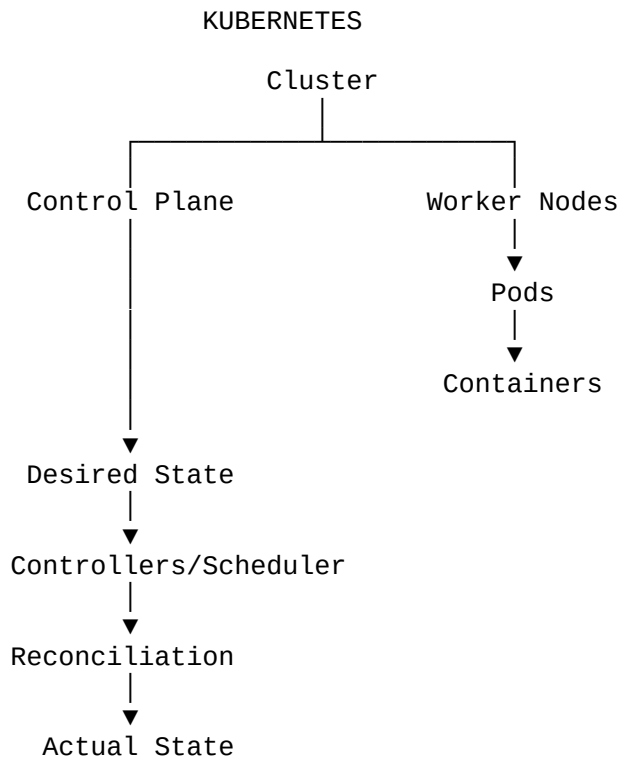
Liveness Probe
Readiness Probe
Startup Probe
Resource Requests
Resource Limits
HPA
Rolling Update
Rollback

Level 7 — Production Kubernetes

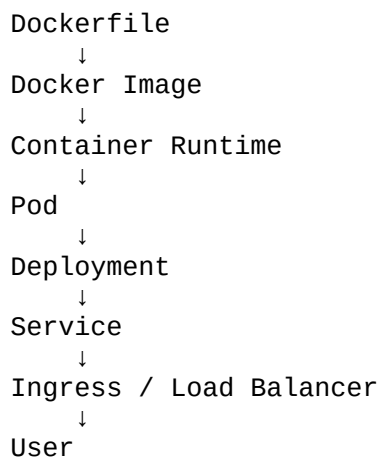
RBAC
ServiceAccount
NetworkPolicy
Helm
Monitoring
Logging
Metrics
Autoscaling
Security
GitOps
CI/CD
Cloud Kubernetes

45. The most important Kubernetes mental model

If you remember only one thing, remember this:



And for deploying an application:



The **key difference from Docker** is that Docker primarily gives you the mechanism to package and run containers, while Kubernetes provides a control system for **declaring, scheduling, networking, scaling, updating, and continuously reconciling containerized workloads across a cluster**.